NASCOMSOFT EMBEDDED HUB | ENGINEERING

# Execution Track: 2-Developer Split

Parallel 7-Day Sprint Guide & GitHub Protocol

To successfully deliver the Network Analyser MVP in 7 days, the workload is split into two parallel tracks: **Backend/Data Pipeline (Engr. Stanley O. Ikegbo)** and **Frontend/UI Dashboards (Glory Zakka)**. Both tracks must synchronize daily to ensure seamless integration.

## 1. GitHub Collaboration Protocol

**Branching & Merging Rules**

- **Main Branch:** `main` is strictly for deployable, working code. No direct commits.
- **Daily Feature Branches:** Use the format `stanley/day1-schema` and `glory/day1-ui-shell`.
- **The 4:00 PM Sync:** By 4:00 PM daily, both developers must open Pull Requests (PRs). Review each other's code and merge into `main` by EOD.
- **API Contracts First:** Before building logic, Stanley must document the JSON structure of API endpoints so Glory can build using mock data immediately.

## 2. Days 1–3: Core Engine & Shell

| Day | Engr. Stanley (Backend) | Glory Zakka (Frontend) |
|---|---|---|
| **Day 1** Architecture | STANLEY Set up GitHub Repo. Initialize backend framework. Define DB Schema (Device, Usage, Health). Draft initial API JSON contracts for UI. | GLORY Clone Repo. Initialize frontend boilerplate. Set up routing, component library, and dummy data based on Stanley's API contracts. |
| **Day 2** Ingestion | STANLEY Build the data ingestion listener. Implement device discovery logic mapping IP to MAC addresses. Save records to DB. | GLORY Build the shell of the Network Usage Dashboard. Create the layout, navigation sidebar, and empty state containers. |
| **Day 3** Telemetry | STANLEY Create APIs for per-device upload/download consumption. Add background jobs for polling network health (latency, speed). | GLORY Implement charting libraries. Build visualizations for bandwidth consumption and the peak-usage timeline using mock data. |

## 3. Days 4–7: Integration & Launch

| Day | Engr. Stanley (Backend) | Glory Zakka (Frontend) |
|---|---|---|
| **Day 4**<br>Logic & Auth | STANLEY Implement background alert triggers (detecting unknown devices, threshold breaches). Expose endpoints for alerts. | GLORY Build Role-Based Access UI (Admin vs Viewer views). Build the Alerts panel and Settings page. |
| **Day 5**<br>The Hookup | STANLEY Build report generation logic (CSV data exports). Assist Glory with CORS or API integration issues. | GLORY **Integration Day:** Swap out all mock data for live API calls to Stanley's endpoints. Ensure loading/error states work. |
| **Day 6**<br>QA & Hardening | STANLEY Perform Privacy Audit (ensure no payloads are stored). Run backend load tests. Fix API bugs identified by Glory. | GLORY Conduct End-to-End UI testing. Fix layout bugs on different screen sizes. Verify privacy notice is prominently displayed. |
| **Day 7**<br>Pilot Launch | SHARED RESPONSIBILITY Deploy the combined `main` branch to the pilot environment. Verify live data ingestion. Hand over credentials to the test administrator. Monitor logs together. | |

**Integration Checkpoints**

Do not wait until Day 5 to integrate. The key to a successful 7-day dual-developer sprint is continuous integration.

- If Stanley finishes an endpoint early, push it to main.
- If Glory needs a data structure changed to make charting easier, communicate it immediately.
- **Merge conflicts are the enemy of speed:** Pull from `main` at the start of every day before writing new code.